

基础类型 & 字节

类型	字节	范围
bool	1	true/false
char	1	−128~127
short	2	−32768~32767
int	4	−2 ^{31} 2 ^{31} − 1
long long	8	−2 ^{63} 2 ^{63} − 1
float	4	7 位有效
double	8	15 位有效
size_t	8	0 2 ^{64} − 1

关键点

整数溢出：int 约 2 × 10⁹，超出用 long long。 **浮点比较**：不能直接 ==，用 eps：

```
#define eps 1e-9
fabs(a - b) < eps
```

数组：局部数组不初始化（值未定义），全局/static 默认零。
除法：整数除法截断，int a = 5/2 → a=2。% 对负数：C++11 向零截断。
未定义行为：a[i] = i++、有符号溢出、解引用空指针。
字符串终止符：string 内部多存一个 \0，strlen 不计入。
宏定义括号：

```
#define SQR(x) ((x)*(x))
// 不加括号：SQR(1+2) → 1+2*1+2 = 5 ≠ 9
```

头文件保护：

```
#ifndef _HEAD_
#define _HEAD_
// ...
#endif
// 或 #pragma once
```

输入输出

```
cin >> a >> b;
cout << x << endl;
scanf("%d %lf", &n, &d);
printf("%.2lf\n", d);
```

加速：ios::sync_with_stdio(false); cin.tie(nullptr);（之后勿混用 scanf/printf）。
argc/argv：int main(int argc, char** argv)，argv[0] 是程序名。
printf 格式：%d %ld %lld %f %lf %s %c %02d %8.3lf

控制流 & 函数

```
int max = (a > b) ? a : b; // 三目
for (int x : arr) { /*...*/ }
for (auto &x : vec) { /*...*/ }
void f(int a, int b = 0); // 默认参数从右往左
```

函数重载：同名不同参数列表（类型/个数），返回值不参与。
声明 vs 定义：声明可多次，定义只能一次。extern int a; 是声明。
内联：inline 建议编译器展开，在头文件中定义。
auto：auto x = expr; 编译期推导类型。auto& 推导引用。
constexpr：constexpr int N = 10; 编译期常量，constexpr int f(int n) { return n*2; }。

指针 & 引用

```
int a = 10;
int *p = &a; // 指针
int &ref = a; // 引用，必须初始化
ref = 20; // a 变为 20
*p = 30; // a 变为 30
```

引用 vs 指针：引用不能为空、不能重绑定、必须初始化。
const 引用：const string& s 可绑定临时对象，避免拷贝。
指针运算：p+1 移动 sizeof(*p) 字节。*(arr+i) ≡ arr[i]。
数组退化：数组传参退化为指针，丢失长度信息。
野指针：delete 后未置 nullptr，悬空指针。

内存管理

```
int *p = new int(5); // 单个
int *arr = new int[100]; // 数组
delete p;
delete[] arr; // [] 不能忘！
```

配对：new ↔ delete，new[] ↔ delete[]。不匹配是 UB。
内存泄漏：new 后未 delete。
malloc/free：C 风格，不调用构造/析构，C++中优先用 new/delete。

位运算

```
& | ^ ~ << >> // 与 或 异或 取反 左移 右移
int bit = (n >> k) & 1; // 获取第 k 位
n |= (1 << k); // 设置第 k 位
n &= ~(1 << k); // 清除第 k 位
if (n & 1) /*奇数*/;
n ^= n; // 快速清零
```

字符串

```
#include <cstring>
strlen(s); // 长度（不含\0）
strcpy(d, s); // 复制（d需足够大）
strcat(d, s); // 追加
strcmp(s1, s2); // 0相等 <0 s1<s2 >0 s1>s2
strchr(s, 'a'); // 查找字符首次出现
strstr(s, "ab"); // 查找子串

#include <string>
string s = "hello";
s.length(); s.size(); // 相同
s += " world"; // 拼接
s.substr(0, 3); // "hel"
s.find("ll"); // 位置或 string::npos
s.c_str(); // 转 const char*
stoi("42"); // string-int
to_string(42); // int-string
getline(cin, s); // 读整行（含空格）
```

STL：常用容器

```
#include <vector>
vector<int> v = {1,2,3};
v.push_back(4); v.pop_back();
v.size(); v.empty(); v.clear();
v[i]; // 不检查越界
v.at(i); // 检查越界

#include <map>
map<string, int> m; // 有序（红黑树）
m["key"] = 42;
m.count("key"); // 0 或 1

#include <unordered_map>
unordered_map<int,int> um; // 哈希表，O(1)查找

#include <set>
set<int> st; st.insert(3); st.count(3);

#include <queue>
queue<int> q; q.push(1); q.front(); q.pop();
priority_queue<int> pq; // 大顶堆
priority_queue<int,vector<int>,greater<>> pq2; // 小顶堆
pq.top(); pq.pop();

#include <stack>
stack<int> sk; sk.push(1); sk.top(); sk.pop();

#include <utility>
pair<int,int> p = {1, 2};
p.first; p.second;
```

STL：算法

```
#include <algorithm>
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<>());
nth_element(v.begin(), v.begin()+k, v.end()); // 第k
swap(a, b); min(a, b); max(a, b);
find(v.begin(), v.end(), val); // 返回迭代器或 end()
count(v.begin(), v.end(), val);
reverse(v.begin(), v.end());
unique(v.begin(), v.end()); // 去重（先sort），返回迭代器
lower_bound(v.begin(), v.end(), val); // ≥val 第一个
upper_bound(v.begin(), v.end(), val); // >val 第一个
binary_search(v.begin(), v.end(), val);
#include <numeric>
accumulate(v.begin(), v.end(), 0); // 求和
```

编译 & 链接 & Make

```
g++ -c main.cpp # 编译→.o
g++ -o main main.o # 链接
g++ -o main main.cpp # 编译+链接
```

分离编译：.h 放声明，.cpp 放定义。
Makefile：

```
all: main
main: main.o stu.o
    g++ -o main main.o stu.o
clean:
    rm -f main *.o
```

头文件改变 → 引用该头文件的源文件全部重编译。

常量 & static & const

```
const int N = 100;           // 必须初始化, 不可修改
static int cnt = 0;          // 生存期=程序全程
                              // 局部 static 只初始化一次
static const int M = 50;     // 全局常量, 内部链接

类中的 static: 属于类而非对象, 所有实例共享。必须在类外定义: int A::cnt = 0;。C++17 可 inline static。类中的 const: 每个对象各有一份。const 成员函数: 不修改对象。

void print() const; // 承诺不修改成员变量

const 位置: const int* p (指向常量) vs int* const p (指针本身是常量) vs const int* const p (都是常量)。
```

OOP: 构造与析构

```
class Obj {
public:
    int x;
    Obj(int a) : x(a) {}           // 初始化列表
    Obj(const Obj& o) : x(o.x) {}  // 拷贝构造
    ~Obj() {}                      // 析构
};
```

初始化列表: 必须用于 const 成员、引用成员、没有默认构造的成员对象。**构造顺序:** 基类 → 成员对象 (按声明序) → 派生类构造函数体。**析构顺序:** 反之: 派生类体 → 成员对象 (反声明序) → 基类。**默认函数:** 不写则自动有: 默认构造、拷贝构造、拷贝赋值、析构。= default 显式保留, = delete 显式删除。**explicit:** 禁止隐式转换构造。

OOP: 继承与派生

```
class Derived : public Base {
public:
    Derived(int a) : Base(a) {} // 显式调用基类构造
};
```

访问权限:

	public	protected	private
public 继承	pub	pro	pri(inacc)
protected 继承	pro	pro	pri(inacc)
private 继承	pri	pri	pri(inacc)

protected: 类内及派生类可访问, 外部不可。**多重继承:** class C : public A, public B {}; **菱形继承:** 两个派生类继承同一基类, 再被多重继承 → 基类成员重复。用虚继承解决: class B : virtual public A {};

OOP: 虚函数与多态

```
class Base {
public:
    virtual void f() { /*...*/ }
    virtual ~Base() {} // 基类析构必须 virtual!
};
class Derived : public Base {
public:
    void f() override { /*...*/ }
};
// 多态
Base* p = new Derived();
p->f(); // Derived::f() (动态绑定)
delete p; // 虚析构确保调 ~Derived
```

纯虚函数: virtual void f() = 0; 使类成为抽象类, 不可实例化。派生类必须实现所有纯虚函数才能实例化。**override:** 编译期检查是否真的重写了基类虚函数。**final:** void f() override final; 阻止进一步重写。class X final {} 阻止继承。**vtable:** 每个含虚函数的类有虚函数表, 对象存指向 vtable 的指针, 运行时查表实现多态。**静态绑定 vs 动态绑定:** 非虚函数看指针/引用类型, 虚函数看实际对象类型。

OOP: 拷贝控制与 char*

深拷贝 vs 浅拷贝: 默认拷贝是浅拷贝 (复制指针值), 含指针成员必须自定义。

```
class PalString {
    char* text;
public:
    PalString(const char* s) {
        text = new char[strlen(s)+1];
        strcpy(text, s);
    }
    PalString(const PalString& o) {
        text = new char[strlen(o.text)+1];
        strcpy(text, o.text);
    }
    PalString& operator=(const PalString& o) {
        if (this != &o) { // 自赋值检查
            delete[] text;
            text = new char[strlen(o.text)+1];
            strcpy(text, o.text);
        }
        return *this;
    }
    ~PalString() { delete[] text; }
};
```

三法则: 需要自定义析构/拷贝构造/拷贝赋值之一, 则三个都需要。

运算符重载

```
class Vec {
    int x, y;
public:
    Vec(int x=0, int y=0) : x(x), y(y) {}
    Vec operator+(const Vec& b) const {
        return Vec(x+b.x, y+b.y);
    }
    Vec& operator+=(const Vec& b) {
        x += b.x; y += b.y;
        return *this;
    }
    Vec& operator++() { ++x; return *this; } // 前置
    Vec operator++(int) { // 后置
        Vec tmp = *this; ++*this; return tmp;
    }
    int& operator[](int i) { return arr[i]; }
    int operator[](int i) const { return arr[i]; }
};
// 全局 << 重载 (必须全局, 左操作数是 ostream)
ostream& operator<<(ostream& os, const Vec& v);
// 全局 + (左操作数不是类类型)
friend CHugeInt operator+(int a, const CHugeInt& b);
```

friend

友元函数/类: 可访问类私有成员, 不属于类, 不受 public/private 限制。

```
class A {
    friend void show(A& a); // 友元函数声明
    friend class B;         // 友元类
private:
    int secret;
};
```

友元不可继承、不可传递。

模板

```
// 函数模板
template <typename T>
T mymax(T a, T b) { return a > b ? a : b; }
mymax(3, 5); // 隐式推导 T=int
mymax<double>(3, 5.0); // 显式指定

// 类模板
template <typename T, int N>
class Stack {
    T data[N]; int top = -1;
public:
    void push(T x) { data[++top] = x; }
    T pop() { return data[top--]; }
};
Stack<int, 100> si;
// 特化
template<> void swap<MyType>(MyType& a, MyType& b);
```

编译期实例化: 模板在编译时根据使用生成具体代码。模板定义通常放头文件中。

类型转换

```
static_cast<int>(d); // 编译期安全转换
static_cast<Derived*>(bp); // 无运行时检查
dynamic_cast<Derived*>(bp); // 运行时检查, 需要虚函数
                              // 失败返回 nullptr(指针)

或抛异常(引用)
const_cast<int&>(cr); // 去除/添加 const
reinterpret_cast<int*>(p); // 底层重解释, 极危险
```

隐式转换: int-double (算术提升)、Derived*~Base* (向上转换)、const char*~string。

struct & union & enum

```
struct Point { // 默认 public
    double x, y;
};
// struct vs class : 仅默认访问权限不同
// union : 所有成员共享同一内存
union Data {
    int i; double d; // 大小 = max(sizeof(int), sizeof(double))
};
// 枚举
enum Color { RED=0, GREEN=1, BLUE=2 };
enum class Dir { UP, DOWN, LEFT, RIGHT }; // C++11 强类型
int x = RED; // OK, 隐式转 int
Dir d = Dir::UP; // 必须用作用域
// int y = Dir::UP; // 错误! 不能隐式转 int
```

命名空间

```
namespace NS {
    int x;
    void f() {}
}
NS::x = 1;
using namespace NS; // 全部引入 (慎用)
using NS::f; // 引入单个
```

异常处理

```
try {
    throw runtime_error("error");
} catch (const exception& e) {
    cout << e.what() << endl;
} catch (...) { // 捕获所有
    cout << "unknown error" << endl;
}
```

杂项

this 指针: 指向当前对象, return *this; 支持链式调用。
mutable: const 函数中仍可修改的成员。sizeof: 编译期求大小。类大小含字节对齐 (按最大成员类型对齐, 通常补到对齐的倍数)。**智能指针 (C++11) :**

```
#include <memory>
unique_ptr<int> p1(new int(5)); // 独占, 不可复制
shared_ptr<int> p2 = make_shared<int>(5); // 引用计数共享
```